APIM AI Gateway — Multi-Foundry Patterns

Routing, Load Balancing, Quota & Throttling for Multi-Model and Multi-Region Foundry Deployments

2026

Contents

0.1 Table of Contents . . . 1
0.2 Overview . . . 2
0.2.1 What APIM AI Gateway adds . . . 2
0.3 Architecture . . . 4
0.4 Pattern 1 — Multi-Endpoint Exposure (Named Backends) . . . 4
0.4.1 Backend definition (Bicep) . . . 5
0.4.2 Routing policy (header-based) . . . 5
0.5 Pattern 2 — Backend Pool with Load Balancing & Circuit Breaker . . . 6
0.5.1 Concept . . . 6
0.5.2 Bicep — pool with circuit breaker . . . 6
0.5.3 Policy . . . 8
0.6 Pattern 3 — Semantic Routing (Preview) . . . 8
0.6.1 Example — route by intent . . . 8
0.6.2 Cost & latency . . . 8
0.7 Centralized Bicep IaC for Multi-Foundry . . . 9
0.7.1 `main.bicep` — full pattern . . . 9
0.7.2 Add an instance — workflow . . . 11
0.8 Quota Flow — How APIM Knows a Backend Is Saturated . . . 11
0.8.1 Sequence — one request, multiple backends . . . 12
0.9 Mid-Chat Throttling — What Happens During a Conversation . . . 12
0.9.1 Three cases . . . 12
0.9.2 Mitigation — continuation pattern . . . 13
0.10 Agent-to-Agent Throttling Patterns . . . 13
0.10.1 1. Budget per conversation, not per call . . . 13
0.10.2 2. Required headers from the orchestrator . . . 13
0.10.3 3. Best-effort vs critical . . . 13
0.10.4 4. Circuit breaker at orchestrator level . . . 14
0.10.5 5. Idempotency keys . . . 14
0.10.6 Sequence — A2A with degradation . . . 14
0.11 Decision Matrix . . . 15
0.12 References . . . 15
0.12.1 Microsoft Learn . . . 15
0.12.2 Sample repositories . . . 15
0.12.3 Companion document . . . 16

Scale **multiple Azure AI Foundry resources** behind a single **Azure API Management (APIM) AI Gateway** — multi-region failover, weighted load balancing, semantic routing, centralized infrastructure-as-code, and resilient quota handling for streaming and agent-to-agent workloads.

0.1 Table of Contents

- Overview
- Architecture
- Pattern 1 — Multi-Endpoint Exposure (Named Backends)

- Pattern 2 — Backend Pool with Load Balancing & Circuit Breaker
- Pattern 3 — Semantic Routing (Preview)
- Centralized Bicep IaC for Multi-Foundry
- Quota Flow — How APIM Knows a Backend Is Saturated
- Mid-Chat Throttling — What Happens During a Conversation
- Agent-to-Agent Throttling Patterns
- Decision Matrix
- References

0.2 Overview

The companion document Foundry_Agent_Monitoring_APIM describes how a **single** Foundry resource can be exposed via APIM for per-user token tracking. This guide answers the next question: **what happens when one Foundry instance is no longer enough?**

Common scaling triggers:

- **Quota exhaustion** — a single AOAI / Foundry deployment hits its TPM (tokens-per-minute) ceiling per region.
- **Multi-region resiliency** — fail over to another region when the primary is throttled or unavailable.
- **Multi-model exposure** — expose `gpt-4o`, `gpt-4o-mini`, `o4-mini`, `text-embedding-3-large`, etc. through one stable URL.
- **Cost optimization** — route cheap prompts to a cheap model, expensive prompts to a frontier model.
- **Tenant / team isolation** — different APIM Products consume different backend pools.

APIM acting as an **AI Gateway** solves all of these without requiring any change in the calling application — clients keep talking to a single URL.

> **Foundry context** — A "Foundry resource" exposes its inference endpoint at `https://<resource>.services.ai.azure.com/api/projects/<project>`. Inside, deployments can be Azure OpenAI, Azure AI Inference, Cohere, Mistral, Llama, or any model from the catalog. APIM treats each one as a backend.

0.2.1 What APIM AI Gateway adds

Capability	Benefit
Backend Pool (GA, October 2024)	Round-robin / weighted / priority load balancing across N backends
Circuit Breaker	Auto-eviction on `429` / `5xx` for a configurable cooldown
`azure-openai-token-limit` **policy**	TPM enforcement per subscription / user / IP
`azure-openai-emit-token-metric` **policy**	Per-caller token + cost metrics in Application Insights

Capability	Benefit
`llm-semantic-cache-lookup/store`	Embedding-based response cache (cuts cost ~30-50% on FAQ workloads)
`llm-semantic-routing` **(preview)**	Route by **prompt meaning**, not by URL or header
Managed Identity	No keys to rotate — APIM authenticates to Foundry with its system-assigned identity

0.3 Architecture

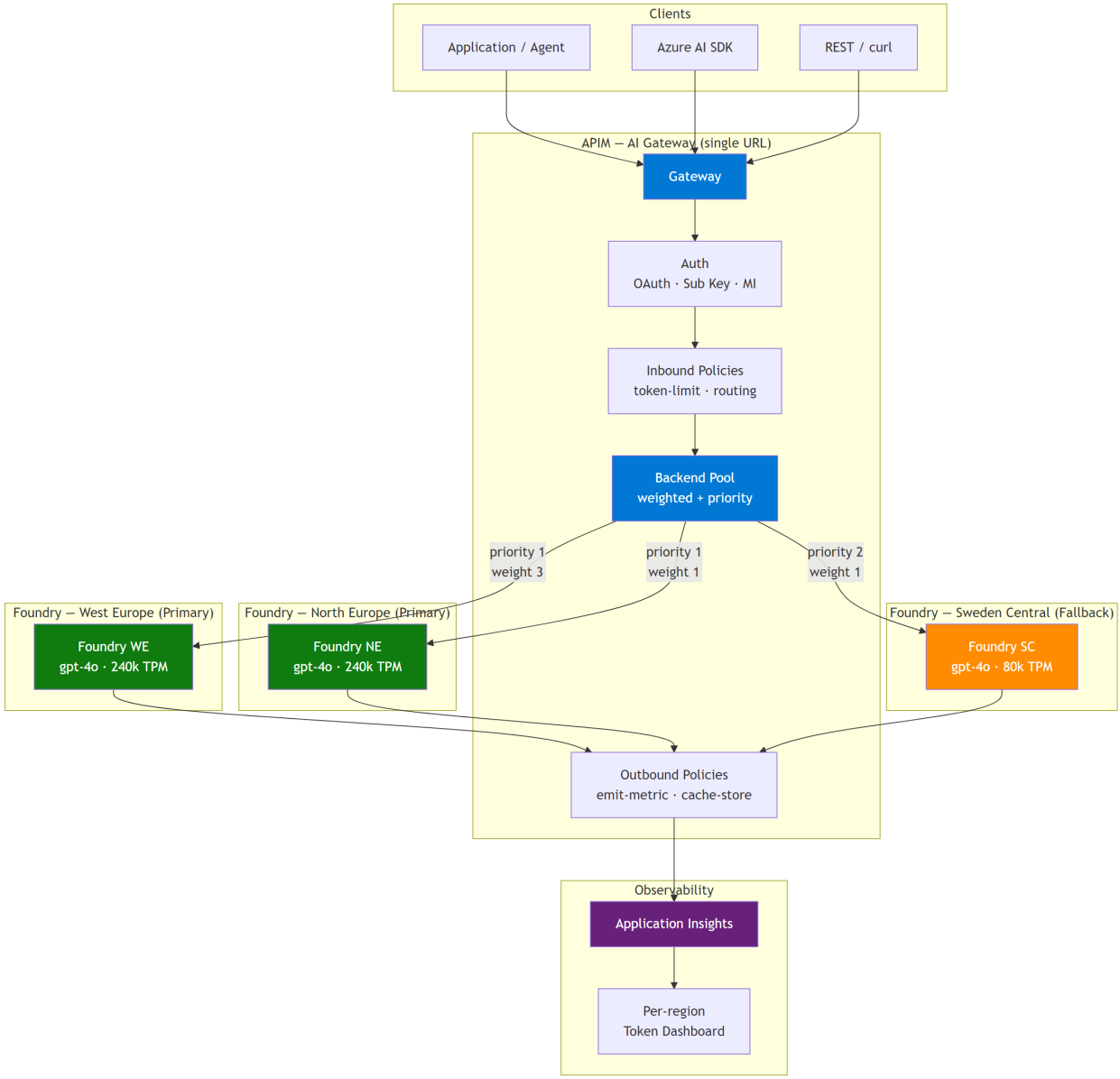

Figure 1: Diagram 1

Key idea — clients see one URL. APIM owns the routing logic, the keys, the metrics, and the resilience. Foundry resources can be added or removed without changing client code.

0.4 Pattern 1 — Multi-Endpoint Exposure (Named Backends)

The simplest pattern. Each model or region is a **named backend**; the routing decision is made by reading a request property (header, query parameter, or body field).

0.4.1 Backend definition (Bicep)

```
resource backendGpt4o 'Microsoft.ApiManagement/service/backends@2024-05-01'
 ↪  = {
  name: 'foundry-gpt-4o'
  parent: apim
  properties: {
    url:
 ↪  'https://foundry-we.services.ai.azure.com/openai/deployments/gpt-4o'
    protocol: 'http'
    credentials: {
      header: { 'api-key': [ '{{foundry-we-key}}' ] }
    }
  }
}

resource backendGpt4oMini
 ↪  'Microsoft.ApiManagement/service/backends@2024-05-01' = {
  name: 'foundry-gpt-4o-mini'
  parent: apim
  properties: {
    url: 'https://foundry-we.services.ai.azure.com/openai/deployments/gpt-
 ↪  4o-mini'
    protocol: 'http'
  }
}
```

0.4.2 Routing policy (header-based)

```
<policies>
  <inbound>
    <base />
    <choose>
      <when condition="@(context.Request.Headers.GetValueOrDefault("x-model
 ↪  ","").Equals("gpt-4o-mini"))">
        <set-backend-service backend-id="foundry-gpt-4o-mini" />
      </when>
      <when condition="@(context.Request.Headers.GetValueOrDefault("x-model
 ↪  ","").Equals("gpt-4o"))">
        <set-backend-service backend-id="foundry-gpt-4o" />
      </when>
      <otherwise>
        <set-backend-service backend-id="foundry-gpt-4o-mini" />
      </otherwise>
    </choose>
  </inbound>
</policies>
```

When to use — small number of models, deterministic routing, no resiliency requirements. Each backend is a single point of failure.

0.5 Pattern 2 — Backend Pool with Load Balancing & Circuit Breaker

The recommended pattern for **resilient multi-region** deployments. APIM's `Pool` backend type combines several backends into one logical target; APIM automatically distributes traffic according to **priority** and **weight** and ejects unhealthy backends via a **circuit breaker**.

0.5.1 Concept

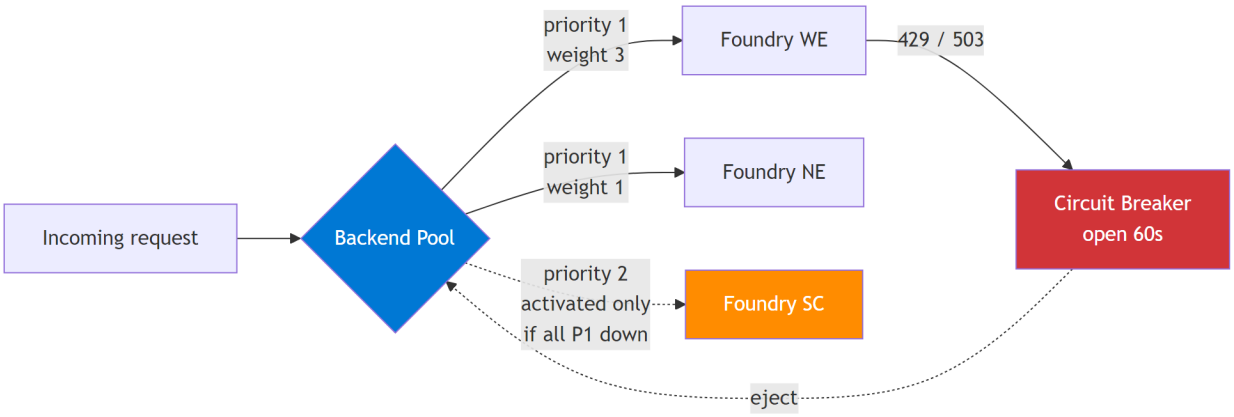

Figure 2: Diagram 2

- **Priority** — backends with priority `1` receive all traffic; priority `2` activates only when all priority-1 backends are tripped.
- **Weight** — within a priority tier, traffic is split proportionally (here `3:1` ratio between WE and NE).
- **Circuit breaker** — after N consecutive failures, the backend is taken out of rotation for a cooldown period. Native trigger on `429` and `5xx`.

0.5.2 Bicep — pool with circuit breaker

```
resource foundryWE 'Microsoft.ApiManagement/service/backends@2024-05-01' = {
  name: 'foundry-we'
  parent: apim
  properties: {
    url: 'https://foundry-we.services.ai.azure.com/api/projects/default'
    protocol: 'http'
    circuitBreaker: {
      rules: [
        {
          name: 'aoai-throttle'
          failureCondition: {
```

```
            count: 3
            interval: 'PT1M'
            statusCodeRanges: [ { min: 429, max: 429 }, { min: 500, max: 599
 ↪  } ]
          }
          tripDuration: 'PT1M'
        }
      ]
    }
  }
}

resource foundryNE 'Microsoft.ApiManagement/service/backends@2024-05-01' = {
  name: 'foundry-ne'
  parent: apim
  properties: {
    url: 'https://foundry-ne.services.ai.azure.com/api/projects/default'
    protocol: 'http'
  }
}

resource foundrySC 'Microsoft.ApiManagement/service/backends@2024-05-01' = {
  name: 'foundry-sc'
  parent: apim
  properties: {
    url: 'https://foundry-sc.services.ai.azure.com/api/projects/default'
    protocol: 'http'
  }
}

resource pool 'Microsoft.ApiManagement/service/backends@2024-05-01' = {
  name: 'foundry-pool'
  parent: apim
  properties: {
    type: 'Pool'
    pool: {
      services: [
        { id: foundryWE.id, weight: 3, priority: 1 }
        { id: foundryNE.id, weight: 1, priority: 1 }
        { id: foundrySC.id, weight: 1, priority: 2 }
      ]
    }
  }
}
```

0.5.3 Policy

```
<policies>
  <inbound>
    <base />
    <set-backend-service backend-id="foundry-pool" />
    <authentication-managed-identity
 ↪  resource="https://cognitiveservices.azure.com" />
  </inbound>
</policies>
```

That's it. APIM handles failover transparently — the client never sees a 429.

0.6 Pattern 3 — Semantic Routing (Preview)

`llm-semantic-routing` (and its Azure-OpenAI-specific cousin `azure-openai-semantic-routing`) routes a request to a backend based on the **embedding similarity** between the prompt and a set of route definitions. It uses an embedding model to convert the user's prompt into a vector, then picks the route whose anchor phrase is closest in cosine distance.

0.6.1 Example — route by intent

```
<policies>
  <inbound>
    <base />
    <azure-openai-semantic-routing
        embeddings-backend-id="text-embedding-3-small"
        threshold="0.75">
      <route backend-id="foundry-codegen">
        <phrase>write code · refactor · debug · explain function</phrase>
      </route>
      <route backend-id="foundry-summarize">
        <phrase>summarize · TL;DR · extract key points</phrase>
      </route>
      <route backend-id="foundry-creative">
        <phrase>write a story · poem · marketing copy · brainstorm</phrase>
      </route>
      <default-route backend-id="foundry-pool" />
    </azure-openai-semantic-routing>
  </inbound>
</policies>
```

0.6.2 Cost & latency

Factor	Impact
Extra embedding call	`text-embedding-3-small` ≈ $0.02 / 1M tokens — negligible for short prompts
Added latency	+50 to +150 ms (one embedding round-trip per request)
Threshold tuning	Below `0.7` → too many false positives; above `0.85` → most requests fall to `<default-route>`

> **Always declare a** `<default-route>` — otherwise unmatched prompts fail with `503`. In preview, also pin a max-results-per-route to avoid pathological hot spots.

0.7 Centralized Bicep IaC for Multi-Foundry

A single declarative file drives the whole topology. Adding a new region or model becomes **a one-line change** — backends, RBAC and the pool regenerate automatically.

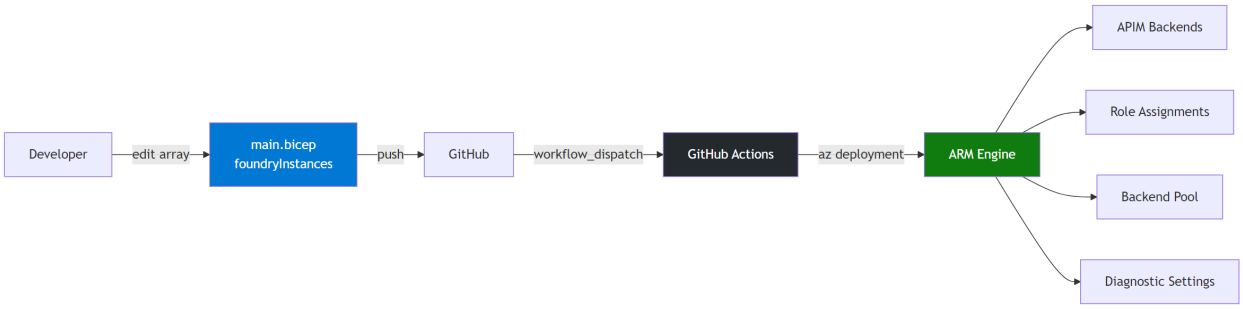

Figure 3: Diagram 3

0.7.1 `main.bicep` — full pattern

```
@description('Declarative list of Foundry instances behind the gateway')
param foundryInstances array = [
  { name: 'foundry-we', region: 'westeurope',     priority: 1, weight: 3 }
  { name: 'foundry-ne', region: 'northeurope',    priority: 1, weight: 1 }
  { name: 'foundry-sc', region: 'swedencentral',  priority: 2, weight: 1 }
]

resource apim 'Microsoft.ApiManagement/service@2024-05-01' existing = {
  name: 'apim-ai-gateway'
}

// 0. Foundry resources (or 'existing' if pre-provisioned)
resource foundryRes 'Microsoft.CognitiveServices/accounts@2024-10-01' = [for
 ↪  f in foundryInstances: {
```

```
  name: f.name
  location: f.region
  kind: 'AIServices'
  sku: { name: 'S0' }
  identity: { type: 'SystemAssigned' }
  properties: { customSubDomainName: f.name }
}]

// 1. Backends auto-created
resource backends 'Microsoft.ApiManagement/service/backends@2024-05-01' =
 ↪  [for (f, i) in foundryInstances: {
  name: 'be-${f.name}'
  parent: apim
  properties: {
    url: 'https://${f.name}.services.ai.azure.com/api/projects/default'
    protocol: 'http'
    circuitBreaker: {
      rules: [ {
        name: 'aoai-throttle'
        failureCondition: {
          count: 3
          interval: 'PT1M'
         statusCodeRanges: [ { min: 429, max: 429 }, { min: 500, max: 599 } ]
        }
        tripDuration: 'PT1M'
      } ]
    }
  }
}]

// 2. RBAC auto-grant — APIM MI gets 'Cognitive Services User' on each
 ↪  Foundry
var cognitiveServicesUserRoleId = 'a97b65f3-24c7-4388-baec-2e87135dc908'

resource roleAssignments
 ↪  'Microsoft.Authorization/roleAssignments@2022-04-01' = [for (f, i) in
 ↪  foundryInstances: {
  name: guid(f.name, apim.id, cognitiveServicesUserRoleId)
  scope: foundryRes[i]
  properties: {
    principalId: apim.identity.principalId
    principalType: 'ServicePrincipal'
    roleDefinitionId:
 ↪  subscriptionResourceId('Microsoft.Authorization/roleDefinitions',
 ↪  cognitiveServicesUserRoleId)
  }
}]
```

```
// 3. Backend Pool - rebuilt on every deployment
resource pool 'Microsoft.ApiManagement/service/backends@2024-05-01' = {
  name: 'foundry-pool'
  parent: apim
  properties: {
    type: 'Pool'
    pool: {
      services: [for (f, i) in foundryInstances: {
        id: backends[i].id
        weight: f.weight
        priority: f.priority
      }]
    }
  }
}
```

0.7.2 Add an instance — workflow

1. Append one row to `foundryInstances`.
2. `git push` → GitHub Actions triggers `az deployment group create`.
3. APIM is up to date — new backend, RBAC, pool, monitoring — **no portal click required**.

Sample reference — the full module (with policies, monitoring and KQL workbooks) is published in the official `Azure-Samples/AI-Gateway` repo.

0.8 Quota Flow — How APIM Knows a Backend Is Saturated

Azure OpenAI / Foundry returns three relevant signals on every response. APIM uses these to decide when to fail over.

Header / Status	Meaning	APIM Reaction
`x-ratelimit-remaining-tokens`	Tokens left in the current TPM window	Informational — surfaced in `azure-openai-emit-token-metric`
`x-ratelimit-remaining-requests`	Requests left in the current RPM window	Informational
`429 Too Many Requests`	Quota exceeded, request rejected	**Circuit breaker counter +1** + Foundry returns `retry-after`
`503 Service Unavailable`	Backend transient failure	**Circuit breaker counter +1**

0.8.1 Sequence — one request, multiple backends

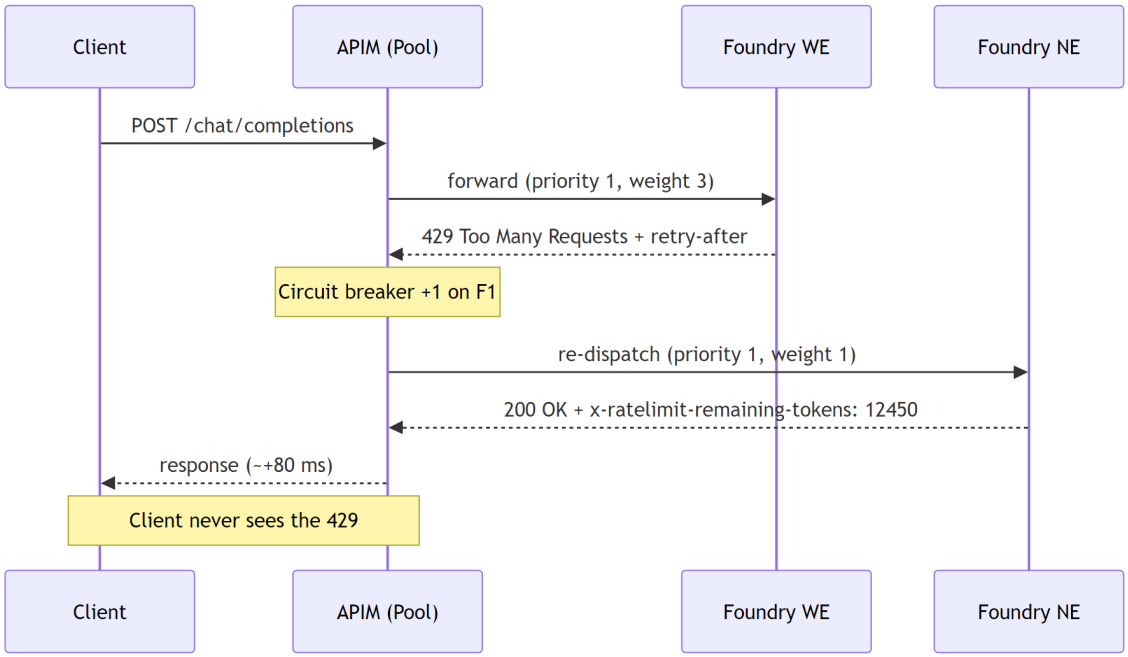

Figure 4: Diagram 4

After three `429`s in 60 seconds, F1 is **opened** for 60 s and all traffic shifts to F2 — and to F3 (priority 2) if both P1 backends are open.

0.9 Mid-Chat Throttling — What Happens During a Conversation

A conversation is **N independent requests** correlated by `thread_id` (Foundry Agents) or by `messages[]` history (raw Chat Completions). APIM re-evaluates routing at **every call** — but a stream that has already started cannot be re-routed.

0.9.1 Three cases

Case	When the 429 lands	APIM behavior	UX impact
Pre-stream	Before the first SSE chunk	APIM swaps backend silently	Invisible (~+80 ms)
Mid-stream	After the first chunk	Stream is **interrupted** with a network-style error	Client must retry — usually with a "continue" prompt
Multi-turn	Between turn N and N+1	Next call routed to a healthy backend	Invisible — conversation continues

0.9.2 Mitigation — continuation pattern

```
def safe_chat(client, messages, max_continuations=2):
    for _ in range(1 + max_continuations):
        try:
            chunks = list(client.chat_stream(messages))
            yield from chunks
            if not is_truncated(chunks): return
            messages.append({"role": "user", "content": "Continue."})
        except RateLimitError:
            time.sleep(1)
```

Foundry's **Threads API** stores conversation state server-side — when a stream fails mid-flight, you re-issue the same `thread_id` and the agent resumes at the last assistant message.

0.10 Agent-to-Agent Throttling Patterns

When agents call other agents (multi-agent orchestration), throttling no longer affects a single user request — it can **cascade** across the whole workflow. Five techniques to keep this under control:

0.10.1 1. Budget per conversation, not per call

```
<azure-openai-token-limit
    counter-key="@(context.Request.Headers.GetValueOrDefault("x-
↪   conversation-id","anon"))"
    tokens-per-minute="60000"
    estimate-prompt-tokens="true"
    remaining-tokens-header-name="x-tokens-remaining" />
```

A 5-agent collaboration with `counter-key="@(thread_id)"` shares the same budget — no agent can starve the others.

0.10.2 2. Required headers from the orchestrator

Header	Purpose
`x-conversation-id`	Token budget scope
`x-agent-id`	Per-agent metric splitting
`x-priority`	`critical` / `best-effort` — used for throttle policies
`x-correlation-id`	End-to-end tracing in App Insights

0.10.3 3. Best-effort vs critical

```
<choose>
  <when condition="@(context.Request.Headers.GetValueOrDefault("x-priority
↪  ",")=="best-effort")">
    <set-backend-service backend-id="foundry-pool-cheap" />
  </when>
  <otherwise>
    <set-backend-service backend-id="foundry-pool-premium" />
  </otherwise>
</choose>
```

Non-critical agents (logging, summarization, reranking) get cheap models; the user-facing answer agent stays on the premium pool.

0.10.4 4. Circuit breaker at orchestrator level

Don't only rely on APIM. The orchestrator (Semantic Kernel, AutoGen, LangGraph…) should track per-agent failure rates and **stop calling** an agent that returned `429` more than X times in T seconds — falling back to a degraded response.

0.10.5 5. Idempotency keys

Every A2A call carries `Idempotency-Key: <uuid>`. APIM caches the response for 5 minutes (`llm-semantic-cache-store`) so retries are free.

0.10.6 Sequence — A2A with degradation

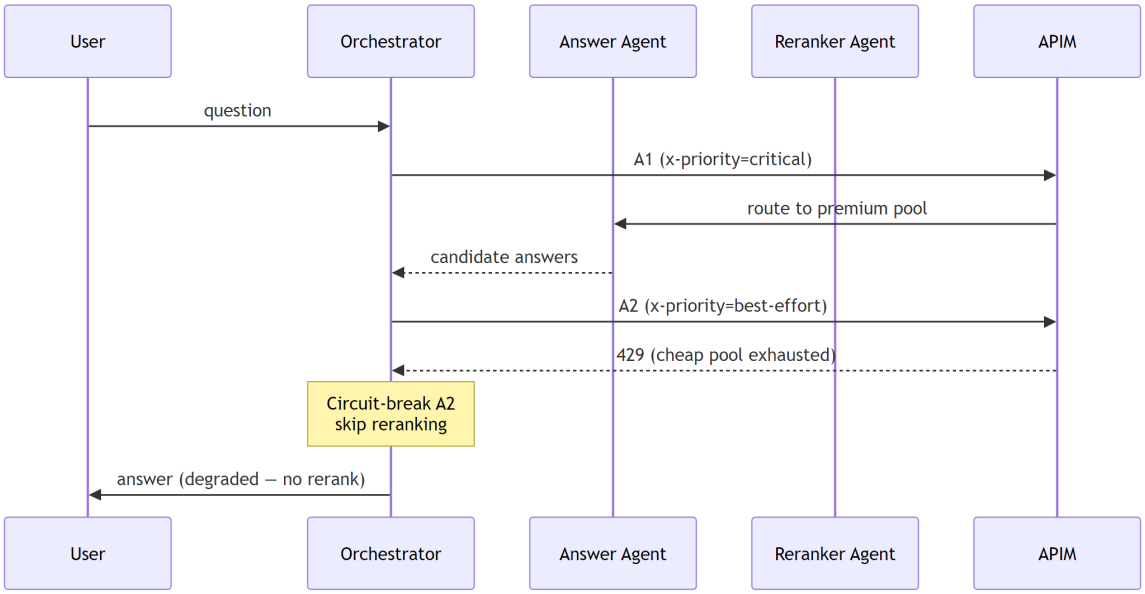

Figure 5: Diagram 5

The user gets a slightly less polished answer, but the system **never returns an error**.

0.11 Decision Matrix

Question	Recommended pattern
One model, one region, < 100 RPS	Direct call to Foundry — no APIM needed
Multiple models exposed via one URL	**Pattern 1** — Named backends + header routing
Multi-region resilience required	**Pattern 2** — Backend Pool + circuit breaker
Cost optimization (route cheap vs frontier)	**Pattern 3** — Semantic routing (preview)
Several Foundry resources to provision and govern	**Centralized Bicep IaC** + GitHub Actions
User-facing chat with intermittent 429s	Pattern 2 + continuation prompts
Multi-agent orchestration with budget	Per-conversation `counter-key` + best-effort priority tier

---

0.12 References

0.12.1 Microsoft Learn

- Backends in API Management — types, pools, circuit breaker
- Backends — load-balanced pool
- Tutorial: load-balance Azure OpenAI traffic
- GenAI Gateway capabilities
- `azure-openai-token-limit` policy
- `azure-openai-emit-token-metric` policy
- `llm-semantic-cache-lookup` policy
- `llm-semantic-routing` policy (preview)
- Authenticate APIM to Azure OpenAI with managed identity
- ARM/Bicep template — `service/backends`
- Bicep loops
- Built-in role — Cognitive Services User
- Azure AI Foundry RBAC
- Foundry quotas, limits & regions

0.12.2 Sample repositories

- `Azure-Samples/AI-Gateway` — full Bicep modules for backend pools, semantic routing, monitoring
  - `labs/backend-pool-load-balancing/`
  - `labs/backend-pool-circuit-breaker/`

- `labs/semantic-routing/`

0.12.3 Companion document

- Foundry_Agent_Monitoring_APIM — single-Foundry per-user token tracking, KQL queries, dashboards